Solution: Serialize and Deserialize Binary Tree

Problem Statement

Given a binary tree, your task is to create two functions.

one for serializing the tree into a string format and another for deserializing the string back into the tree.

The serialized string should retain all the tree nodes and their connections, allowing for reconstruction without any loss of data.

Examples

1. **Example 1:**
   - **Input:** `[1,2,3,null,null,4,5]`
   - **Expected Output:** `[1,2,3,null,null,4,5]`
   - **Justification:** The tree has the structure:

     ```
        1
       / \
      2   3
         / \
        4   5
     ```

     When serialized and then deserialized, it should retain the exact same structure.

2. **Example 2:**
   - **Input:** `[1,null,2,3]`
   - **Expected Output:** `[1,null,2,3]`
   - **Justification:** The tree has the structure:

     ```
        1
         \
          2
         /
        3
     ```

     When serialized and then deserialized, it should retain the exact same structure.

3. **Example 3:**
   - **Input:** `[5,4,7,3,null,null,null,2]`
   - **Expected Output:** `[5,4,7,3,null,null,null,2]`
   - **Justification:** The tree has the structure:

     ```
             5
           /   \
          4     7
         /
        3
       /
      2
     ```

Solution

To serialize a binary tree, we will traverse it in a pre-order fashion (root-left-right) and generate a string representation. When we encounter a `null` value, we'll represent it with a special character, say "X". The serialized string will have each value separated by a comma.

For deserialization, we will split the string on commas and use a queue to help in the reconstruction. We'll take the front of the queue and check if it's "X". If it is, then it's a null node, otherwise, we create a new node with the value. The same approach applies recursively for the left and right children.

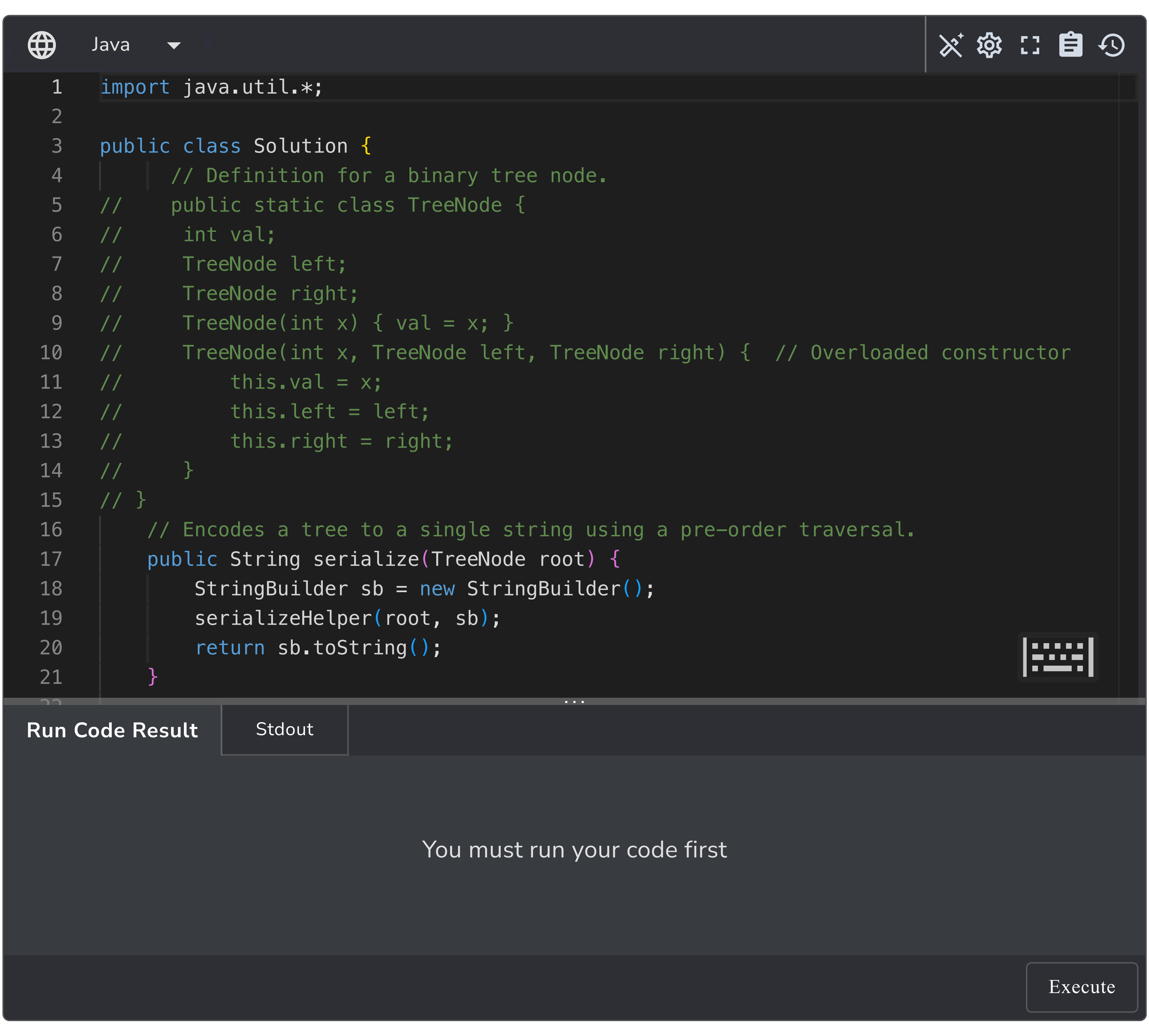
1. **Serialization:**
   - Start from the root.
   - If the current node is null, append "X," to the result string.
   - If not null, append its value and a comma to the result string.
   - Recursively serialize the left subtree.
   - Recursively serialize the right subtree.
2. **Deserialization:**
   - Split the string by comma to get a list of nodes.
   - Use a queue to facilitate tree reconstruction.
   - For each value in the list:
     - If it's "X", return null.
     - Otherwise, create a new node.
     - Recursively deserialize the left and right children.

Algorithm Walkthrough:

Given the input: `[1,2,3,null,null,4,5]`.

- **Serialization:**
  - Start with root: 1. Add "1," to the result string.
  - Go left: 2. Add "2," to the result string.
  - Go left: null. Add "X," to the result string.
  - Go back and go right: null. Add "X," to the result string.
  - Go back to 1 and go right: 3. Add "3," to the result string.
  - Go left: 4. Add "4," to the result string.
  - Left and right of 4 are null. Add "X,X," to the result string.
  - Go back to 3 and go right: 5. Add "5," to the result string.
  - Left and right of 5 are null. Add "X,X," to the result string.
  - **Resulting serialized string:** "1,2,X,X,3,4,X,X,5,X,X,".
- **Deserialization:**
  - Split string by comma: ["1","2","X","X","3","4","X","X","5","X","X"].
  - Start with "1". Create a node with value 1.
  - Move to next value "2". Create a left child with value 2.
  - Next is "X". So, the left of 2 is null.
  - Move to the next "X". Right of 2 is also null.
  - Next is "3". Create a right child for root with value 3.
  - Next is "4". Create a left child for 3 with value 4.
  - Two X's indicate the left and right children of 4 are null.
  - "5" is the right child of 3.
  - Two X's indicate the children of 5 are null.

Code



```java
import java.util.*;

public class Solution {
      // Definition for a binary tree node.
//    public static class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
//     TreeNode(int x, TreeNode left, TreeNode right) {  // Overloaded constructor
//         this.val = x;
//         this.left = left;
//         this.right = right;
//     }
// }
    // Encodes a tree to a single string using a pre-order traversal.
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        serializeHelper(root, sb);
        return sb.toString();
    }
```

Complexity Analysis

Time Complexity

- **Serialization:** We visit every node once and only once, which gives a time complexity of (O(n)), where (n) is the number of nodes in the tree.
- **Deserialization:** Similarly, for deserialization, we reconstruct every node once and only once, so the time complexity is also (O(n)).

Space Complexity

- **Serialization:** In the worst case, we have to append for every node and its two children. Additionally, there might be a considerable number of nulls (`X`), hence the space complexity would be (O(n)).
- **Deserialization:** The primary space consumption lies in the recursion stack, which would be (O(h)), where (h) is the height of the tree. In the worst case, the tree could be skewed, making its height (n), so the space complexity would be (O(n)).

← Previous
Serialize And Deserialize Binary Tree (Hard)

Next →
Where To Go From Here

☐ Mark as Completed